Practical No 01

Aim : Scrape an online E-Commerce Site for Data.

1. Extract product data from Amazon - be it any product and put these details in the MySQL database. One can use a pipeline. Like 1 pipeline to process the scraped data and other to put data in the database and since Amazon has some restrictions on scraping of data, ask them to work on a small set of requests otherwise proxies and all would have to be used.

2. Scrape the details like color, dimensions, material etc. Or customer ratings by features

Code :

```
import requests
from bs4 import BeautifulSoup
import sqlite3

HEADERS = {"User-Agent": "Mozilla/5.0 (X11; Linux x86_64)", "Accept-Language": "en-
US,en;q=0.9"}

conn = sqlite3.connect("amazon_products.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS products (id INTEGER PRIMARY KEY, title
TEXT, price TEXT, rating TEXT)")
conn.commit()

def scrape_amazon(url):
    r = requests.get(url, headers=HEADERS)
    soup = BeautifulSoup(r.text, "html.parser")
    title = soup.find("span", id="productTitle")
    price = soup.find("span", class_="a-price-whole")
    rating = soup.find("span", class_="a-icon-alt")
    return (title.get_text(strip=True) if title else "Not Found",
            price.get_text(strip=True) if price else "Not Found",
            rating.get_text(strip=True) if rating else "Not Found")

title, price, rating = scrape_amazon("https://www.amazon.in/d/B0DZF1485D")
cursor.execute("INSERT INTO products (title, price, rating) VALUES (?, ?, ?)", (title, price, rating))
conn.commit()
print("Data Inserted")
cursor.execute("SELECT * FROM products")
for row in cursor.fetchall():
    print(row)
```

Output :

```
Data Inserted
(1, 'Apple 2025 MacBook Air (13-inch, Apple M4 chip with 10-core CPU and 10-core GPU, 24GB Unified
Memory, 512GB) - Sky Blue', '1,25,490.', '4.7 out of 5 stars')
```

Practical No 02

Aim : Scrape an online Social Media Site for Data.

Code :

```
!pip install -q google-api-python-client

import sqlite3
import re
import pandas as pd
from googleapiclient.discovery import build


API_KEY = 'AIzaSyA-HtWIh09SCy6wQB7W7PAqnBrTjvHYTlw'
URLS = [
    "https://www.youtube.com/watch?v=jNQXAC9IVRw",
    "https://www.youtube.com/watch?v=9bZkp7q19f0"
]
conn = sqlite3.connect("youtube_data.db")
c = conn.cursor()
c.execute("CREATE TABLE IF NOT EXISTS videos (title TEXT, views INT, likes TEXT)")

youtube = build('youtube', 'v3', developerKey=API_KEY)

print("Scraping data...")
for url in URLS:
    try:

        vid_id = re.search(r"(?:v=|\/)([0-9A-Za-z_-]{11})", url).group(1)


        resp = youtube.videos().list(part='snippet,statistics', id=vid_id).execute()

        if resp['items']:
            item = resp['items'][0]
            title = item['snippet']['title']
            views = int(item['statistics'].get('viewCount', 0))
            likes = item['statistics'].get('likeCount', '0')

            c.execute("INSERT INTO videos VALUES (?, ?, ?)", (title, views, likes))
    except Exception as e:
        print(f"Error scraping {url}: {e}")

conn.commit()
```

```
print("\n--- DATABASE CONTENTS ---")
df = pd.read_sql_query("SELECT * FROM videos", conn)
display(df)

conn.close()
```

Output :

```
Scraping data...

--- DATABASE CONTENTS ---
```

	title	views	likes
0	Me at the zoo	379369521	18414723
1	PSY - GANGNAM STYLE(강남스타일) M/V	5815742685	31341419
2	Me at the zoo	379369521	18414722
3	PSY - GANGNAM STYLE(강남스타일) M/V	5815746092	31341418
4	Me at the zoo	379369758	18414717
5	PSY - GANGNAM STYLE(강남스타일) M/V	5815746092	31341419

Practical No 03

Aim : Page Rank for link analysis using python Create a small set of pages namely page1, page2, page3 and page4 apply random walk on the same

Code :

```
import numpy as np

pages = ["page1", "page2", "page3", "page4"]
A = np.array([
    [0, 1, 1, 0],
    [0, 0, 1, 1],
    [1, 0, 0, 1],
    [0, 0, 0, 1]
], dtype=float)

for i in range(len(pages)):
    if A[i].sum() != 0:
        A[i] /= A[i].sum()
    else:
        A[i] = np.ones(len(pages)) / len(pages)

pr = np.ones(len(pages)) / len(pages)
d = 0.85

for _ in range(100):
    pr = (1 - d)/len(pages) + d * np.dot(pr, A)

for i, p in enumerate(pages):
    print(f"{p}: {pr[i]:.4f}")
```

Output :

```
page1: 0.0811
page2: 0.0720
page3: 0.1025
page4: 0.7444
```

Practical No 04

Aim : Perform Spam Classifier

Code :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

data = {
    'message': [
        "Win a free iPhone now", "Hello, how are you?",
        "Congratulations! You won a lottery", "Are we meeting today?",
        "Claim your free prize", "Let's have lunch tomorrow",
        "You have been selected for a gift", "See you at the office"
    ],
    'label': ['spam', 'ham', 'spam', 'ham', 'spam', 'ham', 'spam', 'ham']
}

df = pd.DataFrame(data)
X_train, X_test, y_train, y_test = train_test_split(df['message'], df['label'], test_size=0.25,
random_state=42)

vec = TfidfVectorizer()
X_train_vec = vec.fit_transform(X_train)
X_test_vec = vec.transform(X_test)

clf = MultinomialNB()
clf.fit(X_train_vec, y_train)
y_pred = clf.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Predictions:", clf.predict(vec.transform(["Win a brand new car", "Are you free
tomorrow?"])))
```

Output :

```
Accuracy: 0.0
Predictions: ['spam' 'spam']
```

Practical No 05

Aim : Demonstrate Text Mining and Webpage Pre-processing using meta information from the web pages (Local/Online).

Code :

```
!pip install requests beautifulsoup4 nltk

import requests, re, nltk
from bs4 import BeautifulSoup
from nltk.corpus import stopwords
from collections import Counter

nltk.download('stopwords')

url = "https://www.wikipedia.org/"
soup = BeautifulSoup(requests.get(url).text, "html.parser")

meta = {"title": soup.title.string if soup.title else "N/A"}
for m in soup.find_all("meta"):
    if m.get("name"):
        meta[m.get("name").lower()] = m.get("content")

for t in soup(["script", "style"]):
    t.decompose()

text = soup.get_text(" ").lower()
text = re.sub(r"[^a-z\s]", "", text)

tokens = [w for w in text.split() if w not in stopwords.words("english")]
freq = Counter(tokens)

print("META INFO\n", meta)
print("\nTOP WORDS\n", freq.most_common(10))
```

Output :

```
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-
packages (4.13.5))
META INFO
 {'title': 'N/A'}
TOP WORDS
 [('please', 1), ('set', 1), ('useragent', 1), ('respect', 1), ('robot', 1),
('policy', 1), ('httpswwikiwjs', 1), ('see', 1), ('also', 1),
('httpsphabricatorwikimediaorgt', 1)]
```

Practical No 06

Aim: Apriori Algorithm implementation in case study.

Code :

```
from itertools import combinations

transactions = [
    {"milk", "bread", "butter"},
    {"bread", "butter"},
    {"milk", "bread"},
    {"milk", "butter"},
    {"bread", "butter"}
]

min_support = 2

def get_support(itemset):
    return sum(1 for t in transactions if itemset.issubset(t))

items = set().union(*transactions)
freq_itemsets = []

L1 = [{item} for item in items if get_support({item}) >= min_support]
freq_itemsets.extend(L1)

k = 2
current_L = L1

while current_L:
    candidates = [set(c) for c in combinations(set().union(*current_L), k)]
    next_L = []
    for c in candidates:
        if get_support(c) >= min_support and c not in next_L:
            next_L.append(c)
    freq_itemsets.extend(next_L)
    current_L = next_L
    k += 1

print("Frequent Itemsets:")
for itemset in freq_itemsets:
    print(itemset, "Support:", get_support(itemset))
```

Output :

```
Frequent Itemsets:
{'butter'} Support: 4
{'milk'} Support: 3
{'bread'} Support: 4
{'butter', 'milk'} Support: 2
{'butter', 'bread'} Support: 3
{'milk', 'bread'} Support: 2
```

Practical No 07

Aim: Develop a basic crawler for the web search for user defined keywords.

Code :

```
!pip install requests beautifulsoup4

import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse

keyword = input("Enter keyword to search: ").lower()
start_url = "https://www.wikipedia.org/"
max_pages = 10

visited = set()
to_visit = [start_url]
matched_pages = []

while to_visit and len(visited) < max_pages:
    url = to_visit.pop(0)
    if url in visited:
        continue

    try:
        response = requests.get(url, timeout=5)
        soup = BeautifulSoup(response.text, "html.parser")
        visited.add(url)

        text = soup.get_text(" ").lower()
        if keyword in text:
            matched_pages.append(url)

        for link in soup.find_all("a", href=True):
            next_url = urljoin(url, link["href"])
            if urlparse(next_url).scheme in ["http", "https"]:
                if next_url not in visited and next_url not in to_visit:
                    to_visit.append(next_url)

    except:
        continue

print("\nPages containing keyword:", keyword)
```

```
for page in matched_pages:
    print(page)
```

Output :

```
Requirement already satisfied: requests in
/usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: beautifulsoup4 in
/usr/local/lib/python3.12/dist-packages (4.13.5)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests) (2026.1.4)
Requirement already satisfied: soupsieve>1.2 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4) (2.8.1)
Requirement already satisfied: typing-extensions>=4.0.0 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4) (4.15.0)
Enter keyword to search: rose

Pages containing keyword: rose
```

Practical No 08

Aim : Develop a focused crawler for local search

Code :

```
!pip install beautifulsoup4

import os
from bs4 import BeautifulSoup
from collections import defaultdict

folder_path = "local_pages"
os.makedirs(folder_path, exist_ok=True)

html_files = {
    "page1.html": "<html><body><h1>Rose Wood Furniture</h1><p>Rose wood is used for
premium furniture.</p></body></html>",
    "page2.html": "<html><body><h1>Flower Shop</h1><p>Fresh rose flowers available
here.</p></body></html>",
    "page3.html": "<html><body><h1>Carpentry</h1><p>Quality wood and teak furniture
manufacturing.</p></body></html>"
}

for name, content in html_files.items():
    with open(os.path.join(folder_path, name), "w") as f:
        f.write(content)

keywords = input("Enter keywords (comma separated): ").lower().split(",")

results = defaultdict(int)

for file in os.listdir(folder_path):
    if file.endswith(".html"):
        with open(os.path.join(folder_path, file), "r", encoding="utf-8") as f:
            soup = BeautifulSoup(f.read(), "html.parser")

        text = soup.get_text(" ").lower()
        score = sum(text.count(k.strip()) for k in keywords)
        if score > 0:
            results[file] = score
print("\nFocused Local Search Results:\n")
for page, score in sorted(results.items(), key=lambda x: x[1], reverse=True):
    print(page, "→ Relevance Score:", score)
```

Output :

```
Requirement already satisfied: beautifulsoup4 in
/usr/local/lib/python3.12/dist-packages (4.13.5)
Requirement already satisfied: soupsieve>1.2 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4) (2.8.1)
Requirement already satisfied: typing-extensions>=4.0.0 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4) (4.15.0)
Enter keywords (comma separated): rose

Focused Local Search Results:

page1.html → Relevance Score: 2
page2.html → Relevance Score: 1
```

Practical No 09

Aim: Develop a programme for deep search implementation to detect plagiarism in documents online.

Code :

```
!pip install requests beautifulsoup4 scikit-learn --quiet
import requests
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
text = """Python is a high-level, interpreted programming language known for its simplicity and
readability.
It supports multiple programming paradigms including procedural, object-oriented, and functional
programming.
Python has a rich ecosystem of libraries and frameworks that make it popular for web development,
data analysis,
artificial intelligence, machine learning, and scientific computing. Its syntax emphasizes code
readability,
allowing developers to express concepts in fewer lines of code compared to many other languages.
With a large and active community, Python continues to grow as one of the most widely used
programming languages in the world."""
def search(q): return [a["href"] for a in BeautifulSoup(requests.post("https://duckduckgo.com/html/",
data={"q":q}).text, "html.parser").select("a.result__a", limit=5)]
def get_text(url):
    try:
        s=BeautifulSoup(requests.get(url,timeout=5).text,"html.parser"); [t.decompose() for t in
s(["script","style"])]; return s.get_text(" ")
    except: return ""
def score(d1,d2): return
cosine_similarity(TfidfVectorizer(stop_words="english").fit_transform([d1,d2]))[0][1]
results=[]
for s in [x.strip() for x in text.split(".") if x.strip()][:5]:
    for url in search(s):
        sc=score(text,get_text(url))
        if sc>0.3: results.append((url,round(sc*100,2)))
if results:
    for u,v in sorted(results,key=lambda x:x[1],reverse=True): print(f"{u} → Similarity: {v}%")
else: print("No significant plagiarism detected.")
```

Output :

```
No significant plagiarism detected.
```

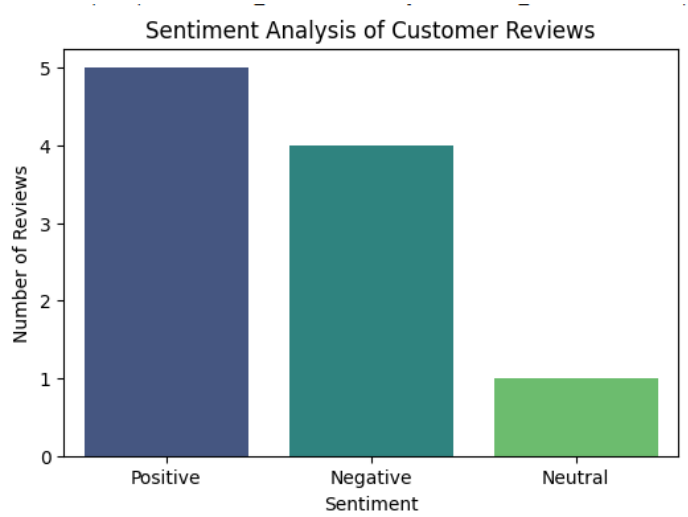
Practical No 10

Aim: Sentiment analysis for reviews by customers and visualize the same.

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer

nltk.download('vader_lexicon')

reviews = [
    "The product is amazing and works perfectly!",
    "Very disappointed with the quality.",
    "Customer service was okay, not great.",
    "Absolutely fantastic experience, highly recommend!",
    "Worst purchase ever. Waste of money.",
    "The item is decent for the price.",
    "I am extremely happy with this product",
    "Not bad, but could be better",
    "Terrible delivery experience",
    "Excellent quality and fast shipping"
]

df = pd.DataFrame({'Review': reviews})

sid = SentimentIntensityAnalyzer()

def get_sentiment(review):
    score = sid.polarity_scores(review)['compound']
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

df['Sentiment'] = df['Review'].apply(get_sentiment)

print(df)

sentiment_counts = df['Sentiment'].value_counts()

plt.figure(figsize=(6,4))
sns.barplot(x=sentiment_counts.index, y=sentiment_counts.values, palette='viridis')
plt.title('Sentiment Analysis of Customer Reviews')
```

```
plt.xlabel('Sentiment')
plt.ylabel('Number of Reviews')
plt.show()
```

Output :

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
                                                 Review Sentiment
0        The product is amazing and works perfectly!  Positive
1               Very disappointed with the quality.  Negative
2             Customer service was okay, not great.  Negative
3  Absolutely fantastic experience, highly recomm...  Positive
4              Worst purchase ever. Waste of money.  Negative
5                 The item is decent for the price.   Neutral
6            I am extremely happy with this product  Positive
7                      Not bad, but could be better  Positive
8                      Terrible delivery experience  Negative
9               Excellent quality and fast shipping  Positive
/tmp/ipython-input-916447292.py:52: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(x=sentiment_counts.index, y=sentiment_counts.values,
palette='viridis')
```